

Documentação da API do Driver Assembly

Co-processador ELM - DE1-SoC - ARMv7 Linux

1. Sobre o Driver

Este documento explica o funcionamento das funções, dos registradores e do protocolo de comunicação do driver desenvolvido para o Marco 2 do projeto. O código foi escrito em Assembly ARMv7 e roda no Linux da placa DE1-SoC, controlando diretamente o coprocessador que está na FPGA.

A comunicação com o hardware é feita por mapeamento de memória no endereço 0xFF200000. Após esse processo, o driver acessa os registradores do coprocessador usando comandos simples de leitura e escrita (LDR e STR), como se fossem posições comuns de memória.

Todas as funções seguem o padrão ARM para organização de subrotinas: os argumentos de entrada entram pelos registradores R0 a R3, o retorno volta pelo R0 e os registradores de R4 a R11 são preservados. O endereço base mapeado fica salvo na variável global `base_mmio` e é carregado no R4 sempre que uma função é iniciada.

2. Registradores e MMIO

2.1 Uso dos Registradores ARM

Registrador	Uso no driver
r0 – r3	Argumentos de entrada e valor de retorno (caller-saved)
r4	Endereço base MMIO (carregado de <code>base_mmio</code>); fd de arquivo em <code>carregar_arquivo</code>
r5	Ponteiro de leitura do buffer no loop; endereço do buffer em <code>carregar_arquivo</code>
r6	Valor do dado atual (int16 após REV16); tamanho do buffer em <code>carregar_arquivo</code>
r7	Número de syscall Linux; cópia do índice nos loops de envio
LR	Endereço de retorno — salvo na pilha em todas as funções via <code>PUSH {..., LR}</code>

2.2 Mapa de Registradores MMIO

Base física: 0xFF200000 - offset de páginas para `mmap2`: 0xFF200

Offset	Registrador	R/W	Descrição
0x00	DATA_OUT	R	Bits[3:0]=dígito predito (0–9) · Bit[4]=DONE · Bit[5]=BUSY · Bit[6]=ERROR

0x10	SIGNALS	W	Bit[0]=enable (pulso strobe) · Bit[1]=clr_operation (limpa DONE) · Bit[2]=rst
0x20	DATA_IN	W	Instrução de 32 bits enviada ao co-processador antes do pulso de enable

3. Protocolo e Conjunto de Instruções

3.1 Protocolo de Handshake

Cada instrução enviada ao co-processador segue a sequência abaixo. O polling é necessário porque a FPGA leva alguns ciclos para processar a instrução e levantar o flag DONE.

Passo	Operação	Registrador afetado
1	Escreve instrução de 32 bits em DATA_IN	STR r0, [r4, #0x20]
2	Ativa enable escrevendo 1 em SIGNALS	STR 1, [r4, #0x10]
3	Lê DATA_OUT em loop até Bit[4] (DONE) = 1	LDR + TST #16 + BEQ
4	Desativa enable escrevendo 0 em SIGNALS	STR 0, [r4, #0x10]

Obs.: STORE_WEIGHTS_ADDR (opcode 001) não executa o passo 3 pois o co-processador retorna ao estado IDLE sem acessar a memória interna e não gera sinal DONE. Para esse caso usa-se enviar_instrucao_sem_done.

3.2 Tabela de Instruções (ISA)

Todas as instruções têm 32 bits. Os bits [2:0] definem o opcode. Os demais campos são posicionados pelas funções de envio.

Instrução	Opcode	Campos	Descrição
STORE_IMG	000	Bits[20:13]=pixel Bits[12:3]=addr	Armazena 1 pixel de 8 bits na posição indicada.
STORE_WEIGHTS_ADDR	001	Bits[19:3]=addr (17 bits)	Define o endereço do próximo peso. Sem DONE.
STORE_WEIGHTS_VALUE	010	Bits[18:3]=valor (16 bits)	Grava o peso no endereço previamente definido.
STORE_BIAS	011	Bits[25:10]=valor Bits[9:3]=addr	Armazena um valor de bias.
STORE_BETA	100	Bits[29:14]=valor Bits[13:3]=addr	Armazena um coeficiente beta.
START	101	—	Inicia a inferência com os dados carregados.

Obs.: Todos os valores de 16 bits (pesos, beta, bias) estão em ponto fixo Q4.12. A instrução REV16 é aplicada após cada LDRH para corrigir a ordem big-endian dos arquivos .bin.

4. Funções Exportadas

4.1 mapear_fpga

Esta função realiza a abertura do arquivo /dev/mem e faz o mapeamento de memória. Isso permite que o programa em C acesse diretamente os registradores físicos da FPGA.

Assinatura C	int mapear_fpga(void)
Entrada	Nenhum
Salva na pilha	r4, r5, r7, lr
Retorno	Endereço virtual mapeado em r0; base_mmio atualizada

Obs.: O offset para mmap2 é dado em unidades de páginas: $0xFF200000 \div 4096 = 0xFF200$. Após essa chamada, todas as demais funções carregam base_mmio em r4 para acessar os registradores.

4.2 reset_coprocessador

Esta subrotina reinicia o coprocessador através de um sinal de reset, limpando o estado interno do hardware para garantir o início seguro da execução.

Assinatura C	void reset_coprocessador(void)
Entrada	Nenhum
Salva na pilha	r4, lr
Retorno	void

4.3 enviar_bias

Esta subrotina abre o arquivo binário de bias, lê os dados para a memória RAM e envia os 128 valores sequencialmente para a FPGA.

Assinatura C	void enviar_bias(const char *caminho)
Entrada	r0 = caminho para b_q.bin
Salva na pilha	r4, r5, r6, r7, lr
Retorno	void

4.4 enviar_beta

Esta subrotina é responsável por ler os coeficientes beta salvos no disco e fazer o envio ordenado de cada um deles para a memória interna do coprocessador.

Assinatura C	void enviar_beta(const char *caminho)
Entrada	r0 = caminho para beta_q.bin
Salva na pilha	r4, r5, r6, r7, lr
Retorno	void

4.5 enviar_img

Esta subrotina faz a leitura dos pixels do arquivo de imagem e envia esses dados para a FPGA na ordem correta para que a rede realize o processamento.

Assinatura C	void enviar_img(const char *caminho)
Entrada	r0 = caminho para o arquivo de imagem
Salva na pilha	r4, r5, r6, r7, lr
Retorno	void

Obs.: Pixels são lidos com LDRB (1 byte). REV16 não é aplicado pois valores de 8 bits não possuem problema de endianness.

4.6 enviar_peso

Esta subrotina realiza o envio da matriz de pesos. Devido ao tamanho dos dados, o envio é dividido em duas partes: primeiro define-se o endereço de gravação e depois entrega-se o valor do peso.

Assinatura C	void enviar_peso(const char *caminho)
Entrada	r0 = caminho para w_in_q.bin
Salva na pilha	r4, r5, r6, r7, lr
Retorno	void

Obs.: Cada peso exige dois envios pois o índice de 17 bits não cabe junto com o valor de 16 bits numa instrução de 32 bits com opcode de 3 bits. O endereço é enviado primeiro sem esperar DONE; o valor na sequência com polling.

4.7 iniciar_inferencia

Esta função inicia o cálculo da classificação no hardware, aguarda a resposta do coprocessador e retorna o dígito previsto de 0 a 9 para o programa em C.

Assinatura C	int iniciar_inferencia(void)
Entrada	Nenhum
Salva na pilha	r4, lr
Retorno	Inteiro 0–9 (dígito predito pela rede ELM)

Obs.: O pulso de clr_operation antes do START limpa um possível DONE stale deixado pela última instrução de envio de dados. Sem essa limpeza o polling poderia sair imediatamente com um resultado inválido.

5. Funções Internas

5.1 carregar_arquivo

Abre um arquivo binário, lê o conteúdo para um buffer em RAM e fecha o arquivo. Chamada por todas as funções de carga de dados.

Entrada	r0=caminho r1=buffer r2=tamanho (bytes)
Salva na pilha	r4, r5, r6, r7, r8, lr
Saída	Não altera r0 de forma relevante

Obs.: r2 é salvo em r6 antes do open() pois o kernel pode modificar r0–r3 durante uma syscall. Sem esse salvamento o tamanho passado para read() seria perdido.

5.2 enviar_instrucao

Envia instrução de 32 bits e bloqueia até DONE = 1. Usada por todas as funções de envio exceto na instrução de endereço de peso.

Entrada	r0=instrução 32 bits r4=base MMIO
Salva na pilha	r1, lr
Saída	Não altera r0

5.3 enviar_instrucao_sem_done

Versão sem polling para STORE_WEIGHTS_ADDR (opcode 001), que não gera sinal DONE.

Entrada	r0=instrução 32 bits r4=base MMIO
Salva na pilha	Nenhum (função folha — BX LR direto)
Saída	Não altera r0

6. Arquivos de Parâmetros

Arquivo	Tamanho	Tipo	Conteúdo
data/w_in_q.bin	200.704 B	int16 Q4.12	100.352 pesos W_in (784 entradas × 128 neurônios)
data/beta_q.bin	2.560 B	int16 Q4.12	1.280 coeficientes beta (128 neurônios × 10 classes)
data/b_q.bin	256 B	int16 Q4.12	128 valores de bias b
data/0.bin ... 99.bin	784 B	uint8	Pixels 28×28 de cada imagem MNIST (1 byte por pixel)

7. Compilação e Execução

O driver é montado como objeto separado e linkado pelo GCC junto com o main.c. O GCC fornece o ponto de entrada e trata main() como início da aplicação.

```
sudo su
as -o driver.o driver.s
gcc -o main main.c driver.o
./main
```

Obs.: sudo é obrigatório para acesso ao /dev/mem. As diretivas .syntax unified e .arm no topo do assembly garantem o interworking correto com o código Thumb-2 gerado pelo GCC.